

PARTITIONING

In Spark

Big Data H/M 2022

Richard McCreadie

SLICING OUR DATASETS

- So far when building our Spark programs, we have been defining transformations on Dataset<Something>
 - However, as you have seen from the Spark Distributed Architecture material, Spark actually splits each dataset into 'partitions' and then processes each partition in parallel
- So how does this happen and can we control it?

HOW MANY PARTITIONS DO WE HAVE?

- The initial partitioning for a Dataset is defined automatically when that dataset is loaded
 - How that happens will depend on what [data loader](#) we used

```
//-----  
// Data Input  
//-----  
// Lets read in some statistics of Steam games from a file  
Dataset<Row> steamGamesAsRowTable = spark  
    .read()  
    .option("header", "true")  
    .csv("data/Steam/games_features.sample.csv");
```

This is a single file
csv reader

- We can get the number of partitions that a dataset has via:
 - `dataset.toJavaRDD().partitions().size()`

Returns an integer
with the number of
partitions

HANDLING FILES

- The csv reader we are using is only working with a single file, and hence has no default way of splitting it into multiple partitions

```
22/02/09 11:56:00 INFO BlockManagerInfo: Added broadcast_3_piece0 in memory on DESKTOP-J91BB8H:53583 (size: 32.6 KiB, free: 2.2 GiB)
22/02/09 11:56:00 INFO SparkContext: Created broadcast 3 from toJavaRDD at Task2Solution.java:71
22/02/09 11:56:00 INFO FileSourceScanExec: Planning scan with bin packing, max size: 4194304 bytes, open cost is considered as scanning 4194
Initial Number of Partitions is: 1
22/02/09 11:56:01 INFO MemoryStore: Block broadcast_4 stored as values in memory (estimated size 61.4 KiB, free 2.2 GiB)
22/02/09 11:56:01 INFO MemoryStore: Block broadcast_4_piece0 stored as bytes in memory (estimated size 4.8 KiB, free 2.2 GiB)
```

- This is of course bad, because even if we have multiple task slots, we will only be able to use one at a time!

MULTIPLE FILES

- Most of the in-built reader types that work on files have a version where you can list multiple files that will form a single dataset

```
//-----  
// Data Input  
//-----  
// Lets read in some statistics of Steam games from a file  
Dataset<Row> steamGamesAsRowTable = spark  
    .read()  
    .option("header", "true")  
    .csv("data/Steam/games_features.1-3000.csv", "data/Steam/games_features.3001-6000.csv", "data/Steam/games_features.6001-9000.csv", "data/Steam/games_features.9001-12000.csv");
```

- Here, each file listed will become its own partition

```
22/02/09 12:08:41 INFO SparkContext: Created broadcast 3 from toJavaRDD at Task2Solution.java:72  
22/02/09 12:08:41 INFO FileSourceScanExec: Planning scan with bin packing, max size: 16711012 bytes, open cost is considered as scanning 4194304 bytes.  
Initial Number of Partitions is: 4  
22/02/09 12:08:41 INFO MemoryStore: Block broadcast_4 stored as values in memory (estimated size 61.4 KiB, free 2.2 GiB)  
22/02/09 12:08:41 INFO MemoryStore: Block broadcast_4_piece0 stored as bytes in memory (estimated size 4.8 KiB, free 2.2 GiB)  
22/02/09 12:08:41 INFO BlockManagerInfo: Added broadcast_4_piece0 in memory on DESKTOP-J91BB8H:56610 (size: 4.8 KiB, free: 2.2 GiB)
```

RE-PARTITIONING

- So what if we don't like the default partitioning when we read the data in?
 - We can **re-partition a dataset** as a transformation function
 - `dataset.repartition(numTargetPartitions);`
- Note that repartitioning is typically an **expensive** operation since it will usually involve **data being moved** to different workers
 - However, that cost may be worth it if we can increase parallelism (max out our task slot usage)
- It is recommended that a dataset is partitioned into **around double or triple the number of task slots you have available**
 - This can help balance load if your partitions don't all take the same amount of time to complete (typically because they don't have the same number of items)

HASH PARTITIONER

- Ok, so re-partition will break a dataset into a new set of partitions, but how does this happen?
 - By default, Spark will use the `HashPartitioner`, this calculates a hash key for each item which resolves to a number *between* $0 < \text{HashKey} < \text{numPartitions}$
 - Partition assignment then is based on the generated key
- Does this always do what we want it to... `no`
 - Usually we want to evenly spread our items across partitions,
 - HashPartitioner is `not guaranteed to do this`,
 - e.g. if your items are strings, the partitioning will be random, leading to unbalanced partitions

CUSTOM PARTITIONIER

- As a transformation function, we can define our own custom logic for partitioning
 - At its core a Partitioner simply takes in a key and returns an integer representing the partition the associated item should be sent to

```
package uk.ac.gla.dcs.bigdata.functions.partition;

import org.apache.spark.Partitioner;

/**
 * A custom partitioning function for use on Strings
 * @author Richard
 */
public class StringPartitioner extends Partitioner{

    private static final long serialVersionUID = -3926494224692694945L;

    final int numberOfPartitions;

    public StringPartitioner(int numberOfPartitions) {
        this.numberOfPartitions = numberOfPartitions;
    }

    @Override
    public int getPartition(Object key) {
        // The object provided as input is whatever the item type is of the dataset

        // for example, if our items were Strings
        String item = (String)key;

        // if we wanted to partition based on the first character of the String
        int partition = item.charAt(0) % numberOfPartitions;

        return partition;
    }

    @Override
    public int numPartitions() {
        return numberOfPartitions;
    }
}
```

CUSTOM PARTITIONIER

- Unfortunately, custom partitioning is not supported by Datasets, rather its is lower level functionality reserved for RDDs
 - This is one of the very few cases where you might need to use something other than a Dataset.

Start from our Dataset

```
Dataset<SteamGameStats> steamGames = steamGamesAsRowTable.map(new SteamStatsFormatter(), Encoders.bean(SteamGameStats.class));
```

```
// convert to RDD
```

```
JavaRDD<SteamGameStats> steamGamesAsRDD = steamGames.toJavaRDD();
```

Convert to an RDD

```
// Partitioners need <key,item> pairs, so we need to extract a key
```

```
GameTitlePairing titleKeyExtractor = new GameTitlePairing(); // extracts a title as a key, returning a Tuple2<String,SteamGameStats>
```

```
JavaPairRDD<String,SteamGameStats> steamGamesWithTitleKeysAsRDDPair = steamGamesAsRDD.mapToPair(titleKeyExtractor); // JavaPairRDD is like a KeyValueGroupedDataset
```

```
// Re-Partition the RDD
```

```
JavaPairRDD<String,SteamGameStats> steamGamesRepartitionedAsRDDPair = steamGamesWithTitleKeysAsRDDPair.partitionBy(new StringPartitioner(12));
```

```
// Drop the keys now we don't need them any more
```

```
JavaRDD<SteamGameStats> steamGamesRepartitionedAsRDD = steamGamesRepartitionedAsRDDPair.map(x -> x._2);
```

```
// Convert back to a Dataset
```

```
Dataset<SteamGameStats> steamGamesRepartitioned = spark.createDataset(steamGamesRepartitionedAsRDD.rdd(), Encoders.bean(SteamGameStats.class));
```

Get convert to a key-paired RDD, as partitioning looks at the key (not value)

PARTITIONIER

Datasets, rather its is
to use something other

- Un
- low
-

```
package uk.ac.gla.dcs.bigdata.functions.pairfunction;

import org.apache.spark.api.java.function.PairFunction;

/**
 * Converts a SteamGameStats to a Tuple2<String,SteamGameStats> where the string is the game title.
 * @author Richard
 */
public class GameTitlePairing implements PairFunction<SteamGameStats, String, SteamGameStats> {

    private static final long serialVersionUID = 5220459648385585521L;

    @Override
    public Tuple2<String, SteamGameStats> call(SteamGameStats t) throws Exception {

        Tuple2<String, SteamGameStats> pairing = new Tuple2<String, SteamGameStats>(t.getTitle(), t);

        return pairing;
    }
}
```

Start from our
Dataset

Get convert to a key-paired
RDD, as partitioning looks at the
key (not value)

```
Dataset<SteamGameStats> steamGames = steamGamesAsRowTable.map(new SteamStatsFormatter(), Encoders.bean(SteamGameStats.class));

// convert to RDD
JavaRDD<SteamGameStats> steamGamesAsRDD = steamGames.toJavaRDD();

// Partitioners need <key,item> pairs, so we need to extract a key
GameTitlePairing titleKeyExtrator = new GameTitlePairing(); // extrats a title as a key, returning a Tuple2<String,SteamGameStats>
JavaPairRDD<String,SteamGameStats> steamGamesWithTitleKeysAsRDDPair = steamGamesAsRDD.mapToPair(titleKeyExtrator); // JavaPairRDD is like a KeyValueGroupedDataset

// Re-Partition the RDD
JavaPairRDD<String,SteamGameStats> steamGamesRepartitionedAsRDDPair = steamGamesWithTitleKeysAsRDDPair.partitionBy(new StringPartitioner(12));

// Drop the keys now we don't need them any more
JavaRDD<SteamGameStats> steamGamesRepartitionedAsRDD = steamGamesRepartitionedAsRDDPair.map(x -> x._2);

// Convert back to a Dataset
Dataset<SteamGameStats> steamGamesRepartitioned = spark.createDataset(steamGamesRepartitionedAsRDD.rdd(), Encoders.bean(SteamGameStats.class));
```

Convert to an RDD

CUSTOM PARTITIONIER

- Unfortunately, custom partitioning is not supported by Datasets, rather its is lower level functionality reserved for RDDs
 - This is one of the very few cases where you might need to use something other than a Dataset.

```
Dataset<SteamGameStats> steamGames = steamGamesAsRowTable.map(new SteamStatsFormatter(), Encoders.bean(SteamGameStats.class));  
  
// convert to RDD  
JavaRDD<SteamGameStats> steamGamesAsRDD = steamGames.toJavaRDD();  
  
// Partitioners need <key,item> pairs, so we need to extract a key  
GameTitlePairing titleKeyExtrator = new GameTitlePairing(); // extrats a title as a key, returning a Tuple2<String,SteamGameStats>  
JavaPairRDD<String,SteamGameStats> steamGamesWithTitleKeysAsRDDPair = steamGamesAsRDD.mapToPair(titleKeyExtrator); // JavaPairRDD is like a KeyValueGroupedDataset  
  
// Re-Partition the RDD  
JavaPairRDD<String,SteamGameStats> steamGamesRepartitionedAsRDDPair = steamGamesWithTitleKeysAsRDDPair.partitionBy(new StringPartitioner(12));  
  
// Drop the keys now we don't need them any more  
JavaRDD<SteamGameStats> steamGamesRepartitionedAsRDD = steamGamesRepartitionedAsRDDPair.map(x -> x._2);  
  
// Convert back to a Dataset  
Dataset<SteamGameStats> steamGamesRepartitioned = spark.createDataset(steamGamesRepartitionedAsRDD.rdd(), Encoders.bean(SteamGameStats.class));
```

Start from our Dataset

Convert to an RDD

Get convert to a key-paired RDD, as partitioning looks at the key (not value)

Apply our custom partitioner

Drop the key (this is an example of a short-hand MapFunction definition using a lambda expression)

Switch back to the dataset